KICSIT

Practical Implementation Report

Sentiment Analyzer Android Application
Kotlin and TensorFlow Lite

Course	Mobile App Development
Instructor	Sir Uzair Hassan
Institution	KICSIT
Submitted by	Fatima Tehreem
Roll Number	242201073
Date	10/31/2026

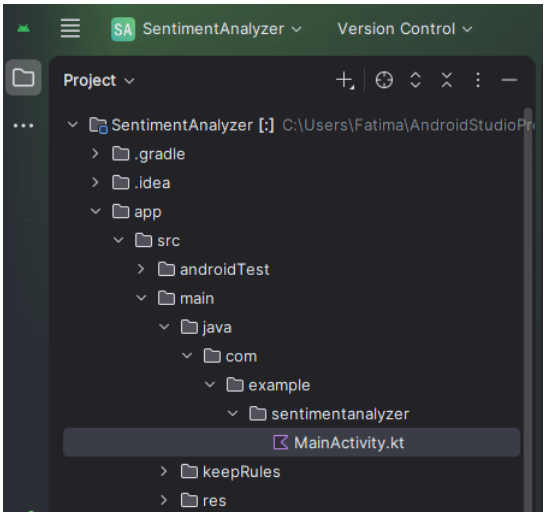
1. Project Overview

SentimentAnalyzer is an Android application written in Kotlin that classifies the sentiment of text typed by the user. A TensorFlow Lite (TFLite) model bundled inside the app performs the classification on the device. After the user taps *Analyze Sentiment*, the app shows an emoji, a sentiment title, the model's label and a confidence percentage. This report covers the project setup, dependencies, model loading, input processing, inference, output, testing and run instructions. The full source code is submitted separately as the Code Snippets document.

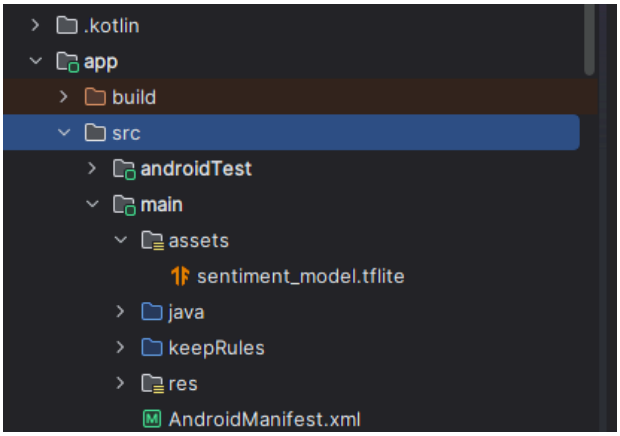
2. Project Setup

2.1 Project Structure

The project is a single-module Android Studio project. The application code is in `MainActivity.kt` under the package `com.example.sentimentanalyzer` (Figure 1a). The model file `sentiment_model.tflite` (751 KB) is stored in `app/src/main/assets` (Figure 1b).



(a) MainActivity.kt in the package



(b) sentiment_model.tflite in assets

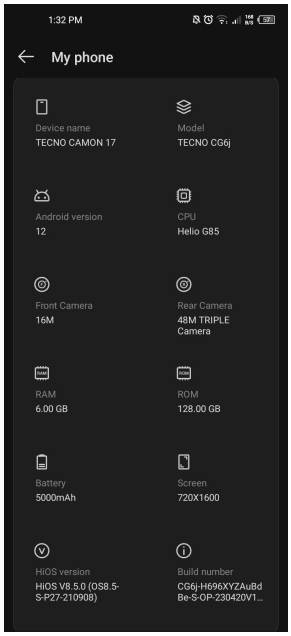
Figure 1: *Project structure of SentimentAnalyzer.*

2.2 Development and Test Environment

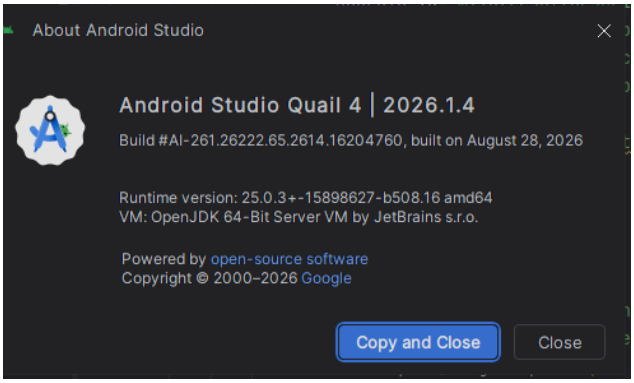
The app was built in Android Studio Quail 4 | 2026.1.4 (Figure 2b) and run on a TECNO CAMON 17 phone running Android 12 (Figure 2a). Table 1 lists the details.

Table 1: Development and test environment

Item	Value
IDE	Android Studio Quail 4 | 2026.1.4 (build AI-261.26222.65.2614.16204760)
Test device	TECNO CAMON 17 (model TECNO CG6j)
Android version / OS skin	Android 12 / HiOS V8.5.0
CPU / RAM / storage	Helio G85 / 6.00 GB / 128.00 GB
Screen	720 x 1600



(a) Phone details (Settings > My phone)



(b) Android Studio version (Help > About)

Figure 2: *Test device and development environment.*

2.3 Build Configuration

Figure 3 shows the top of `build.gradle.kts` and Table 2 summarises the configuration.

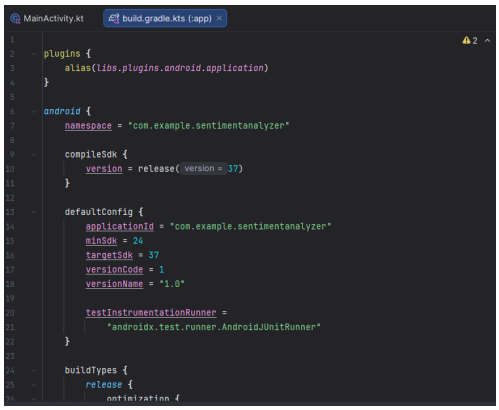


Figure 3: *build.gradle.kts (:app): namespace, compileSdk and defaultConfig.*

Table 2: Build configuration

Item	Value
Language / build script	Kotlin / Gradle Kotlin DSL
Namespace and applicationId	com.example.sentimentanalyzer
compileSdk / minSdk / targetSdk	37 / 24 / 37
versionCode / versionName	1 / "1.0"
Android Gradle Plugin	9.4.1
Java compatibility	VERSION_11
Model file	sentiment_model.tflite, 751 KB

3. Dependencies

Dependencies are declared in `build.gradle.kts` (Figure 4). The key addition is `org.tensorflow:tensorflow-lite-task-text:0.4.4`, which provides the `NLClassifier` class used for text classification. The other entries come from the version catalog and support the user interface and testing (Table 3).

```
}

dependencies {
    implementation(libs.androidx.activity.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.androidx.core.ktx)
    implementation(libs.material)

    // TensorFlow Lite text classification
    implementation("org.tensorflow:tensorflow-lite-task-text:0.4.4")

    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
}
```

Figure 4: *The dependencies block of build.gradle.kts.*

Table 3: Dependencies and versions

Dependency	Version	Purpose
tensorflow-lite-task-text	0.4.4	TFLite text classification (NLClassifier)
androidx.appcompat	1.8.0	AppCompatActivity and compatible UI
com.google.android.material	1.14.0	Material Design components
androidx.activity-ktx / core-ktx	1.13.0 / 1.19.1	Kotlin extensions
androidx.constraintlayout	2.2.2	Layout support
JUnit / AndroidX JUnit / Espresso	4.13.2 / 1.3.0 / 3.7.0	Unit and UI testing

4. User Interface

The screen is defined in `activity_main.xml` (Figure 5). A scrolling layout contains a title, an input card with a 150dp multi-line text box (maximum 500 characters) and the *Analyze Sentiment* button, a result card, and a footer reading “Powered by TensorFlow Lite”. The result card holds three text views: `resultEmoji`, `resultText` and `resultDescription`. Their initial text is a speech-bubble emoji, “Your result will appear here” and “Enter some text above to get started.”

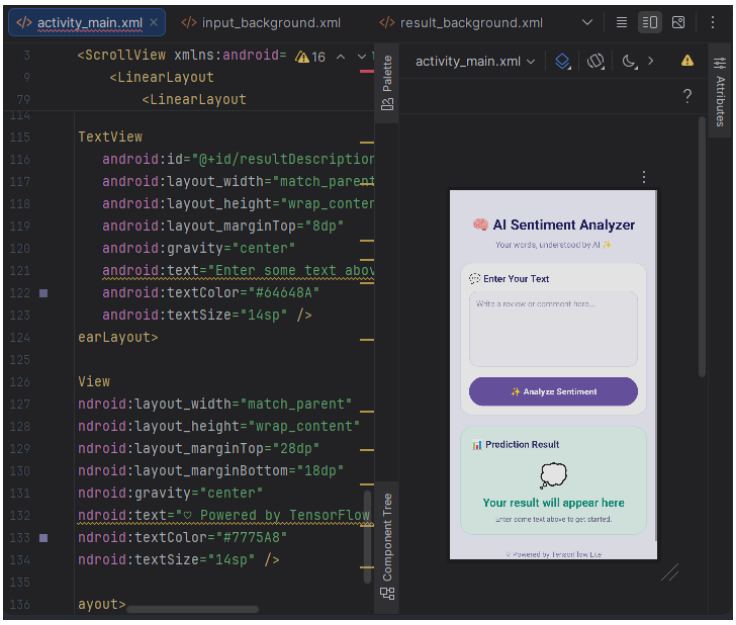


Figure 5: *activity_main.xml code with the design preview of the screen.*

5. Model Loading

The classifier is stored in a nullable property, `classifier`. In `onCreate`, after the views are located, `NLClassifier.createFromFile` loads `"sentiment_model.tflite"` from the assets folder (Figure 6). If loading fails, a Toast reports “Model could not load” with the error message and the classifier stays null. Because the model is bundled with the app, inference runs on the device without a network connection.

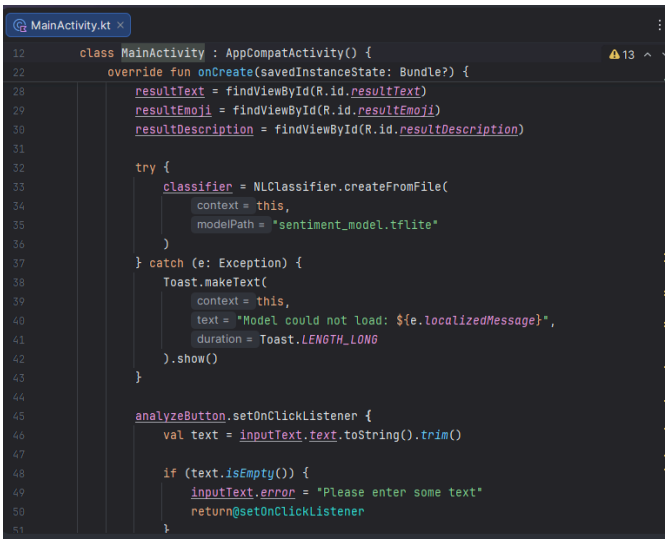


Figure 6: *MainActivity.kt: model loading with error handling, and the start of the click listener.*

6. Input Processing

When the button is tapped, the click listener (Figure 6, lines 45 to 51) reads the text with `inputText.text.toString().trim()`. If it is empty, the message “Please enter some text” is set as an error on the text box and the listener returns without running the model. Otherwise the trimmed string is passed directly to the classifier; no extra tokenisation code is needed.

7. Inference

The listener copies the classifier into a local `model` variable. If it is null, the app displays “Model not loaded”. Otherwise, `model.classify(text)` returns a list of results (Figure 7). An empty list shows “No prediction”, and `maxByOrNull { it.score }` selects the result with the highest score; if none is found, the app asks the user to try another sentence.

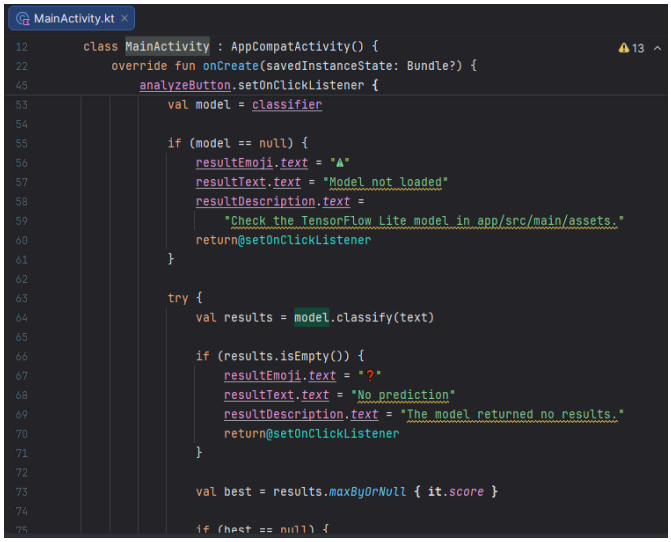


Figure 7: *MainActivity.kt: classifier check, classify call and empty-results handling.*

8. Output Processing and Display

From the best result, the label is trimmed and lower-cased, and the confidence is calculated as `(score * 100).toInt()` (Figure 8). A `when` expression then maps the label to the emoji, title and title colour (Table 4, Figures 8 and 9). The description shows “Model label” and “AI confidence” on two lines. A `catch` block shows “Prediction failed” if an exception occurs, and `onDestroy` closes the classifier (Figure 10).



Figure 8: *MainActivity.kt: best result, label, confidence and the positive branch.*



Figure 9: *MainActivity.kt: negative and else branches and the description text.*

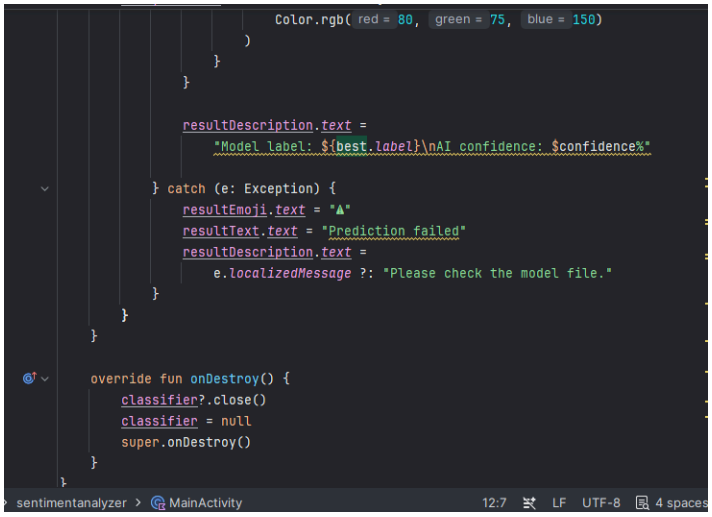


Figure 10: *MainActivity.kt: error handling and onDestroy.*

Table 4: Mapping from model label to display

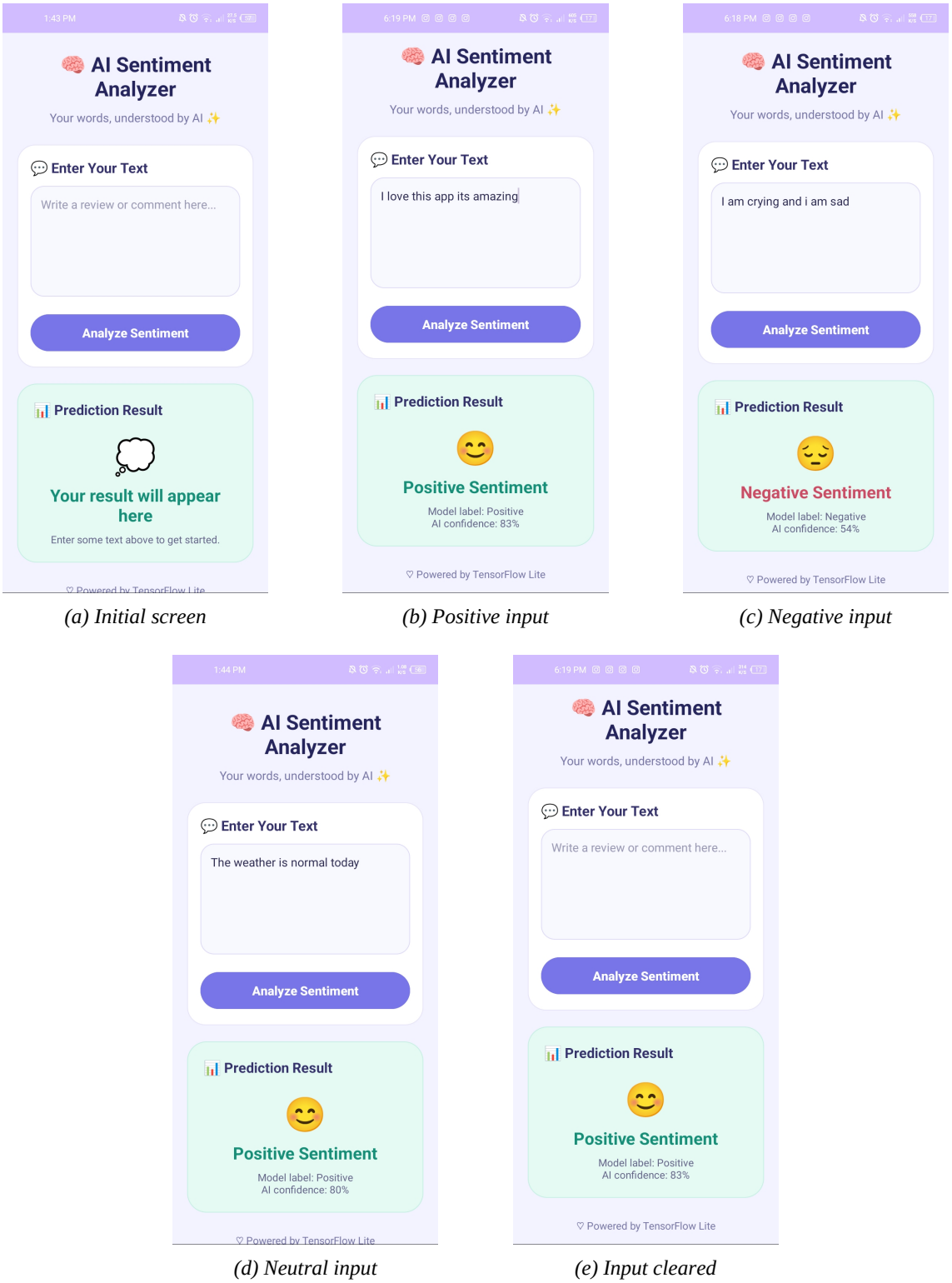
Label condition	Title	Title colour (RGB)
contains "positive", or equals "pos" or "1"	Positive Sentiment	(22, 143, 120)
contains "negative", or equals "neg" or "0"	Negative Sentiment	(205, 75, 100)
any other label	Model Result	(80, 75, 150)

9. Testing

The application was tested on the TECNO CAMON 17 running Android 12 (Table 1). Table 5 lists the five tests and the results observed in the screenshots (Figure 11).

Table 5: Test cases and observed results

#	Test	Input	Observed result	Fig.
1	Initial screen	(none)	Default result card: “Your result will appear here”	11(a)
2	Positive	“I love this app its amazing”	Positive Sentiment, confidence 83%	11(b)
3	Negative	“I am crying and i am sad”	Negative Sentiment, confidence 54%	11(c)
4	Neutral	“The weather is normal today”	Positive Sentiment, confidence 80%	11(d)
5	Input cleared	(empty box)	Hint shown; earlier result (Positive, 83%) remains on the card	11(e)



(a) Initial screen *(b) Positive input* *(c) Negative input*

(d) Neutral input *(e) Input cleared*

Figure 11: *The Sentiment Analyzer running with the five test inputs of Table 5.*

The clearly positive and clearly negative sentences were classified correctly. The neutral sentence was reported as Positive; only Positive and Negative results appear in the screenshots. These few tests do not measure accuracy, so no accuracy figure is claimed.

10. Run Instructions

Requirements: Android Studio with the Android SDK for API 37, an internet connection for the first Gradle sync, and a device or emulator running Android 7.0 (API 24) or higher.

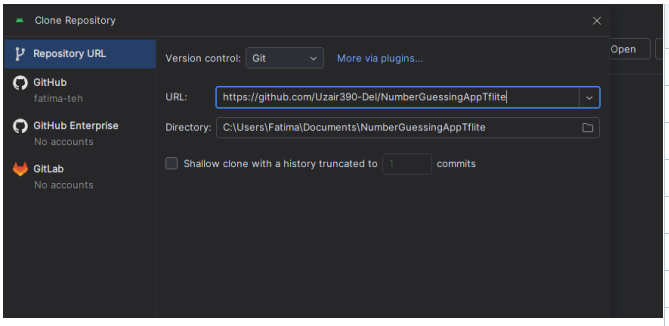
1. Open Android Studio, choose **File > Open** and select the `SentimentAnalyzer` project folder.
2. Wait for the Gradle sync to finish (click **Sync Now** if prompted).
3. Check that `sentiment_model.tflite` is in `app/src/main/assets`.
4. Select a connected device or an emulator and click **Run** for the **app** configuration.
5. Type a sentence, tap **Analyze Sentiment** and read the emoji, sentiment, model label and confidence.

11. Reference Project

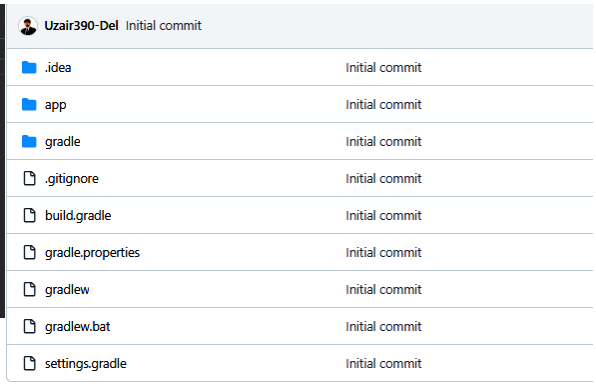
The reference is the original GitHub project *Number Guessing TFLite*: https://github.com/Uzair390-Del/NumberGuessingAppTflite. It is a separate project and is not part of the SentimentAnalyzer implementation above.

11.1 Cloning

The project was cloned in Android Studio with the *Clone Repository* dialog: version control Git, the repository URL above, and a local directory named NumberGuessingAppTflite under Documents (Figure 12a). Figure 12b shows the repository root on GitHub.



(a) Clone Repository dialog in Android Studio

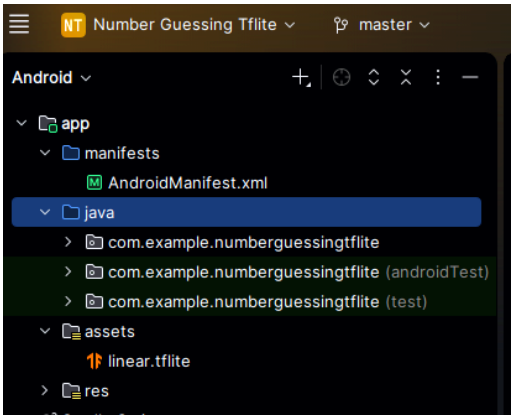


(b) GitHub repository root

Figure 12: *Cloning the reference project.*

11.2 Structure and Code

It is a Java app with the model file `linear.tflite` in the assets folder (Figure 13a). Its `MainActivity.java` loads the model with an `Interpreter` through a `loadModelFile()` method that memory-maps the asset. On a button click it parses a float from the text box, runs `interpreter.run(input, output)` and shows the output value (Figure 14).



(a) Structure in Android Studio



(b) App running: input 3, output 5.0001836

Figure 13: *The reference project: structure and execution.*

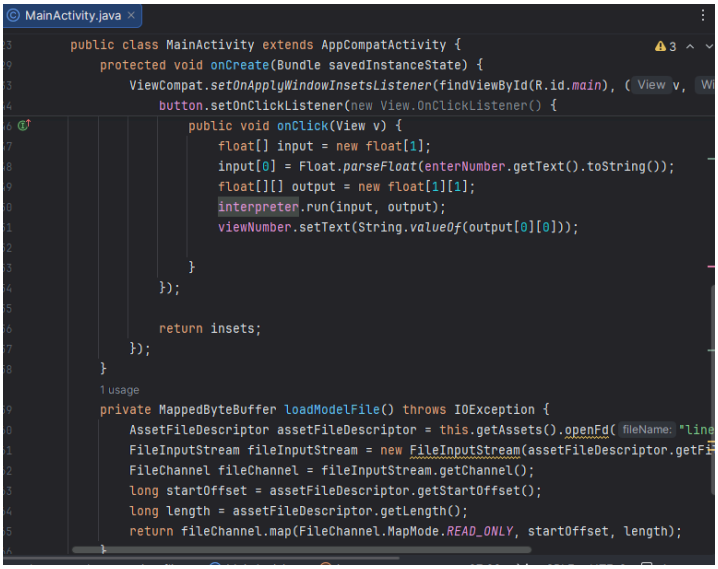


Figure 14: *Reference project: button click inference and the loadModelFile() method.*

11.3 Execution

Figure 13b shows the reference app running on a phone: the title is “Number Guessing Tflite”, the input is 3, and after tapping SEND the app displays 5.0001836. SentimentAnalyzer is a different app that classifies text with its own model through the Task Library's `NLClassifier`.

12. Conclusion

SentimentAnalyzer loads a bundled TFLite model with `NLClassifier`, validates the user's text, classifies it on the device, and displays the sentiment with a confidence percentage. Error cases (model not loaded, empty input, no result, prediction failure) are handled with clear messages, and the classifier is closed when the activity is destroyed. On the test device, positive and negative sentences were classified in line with their meaning.